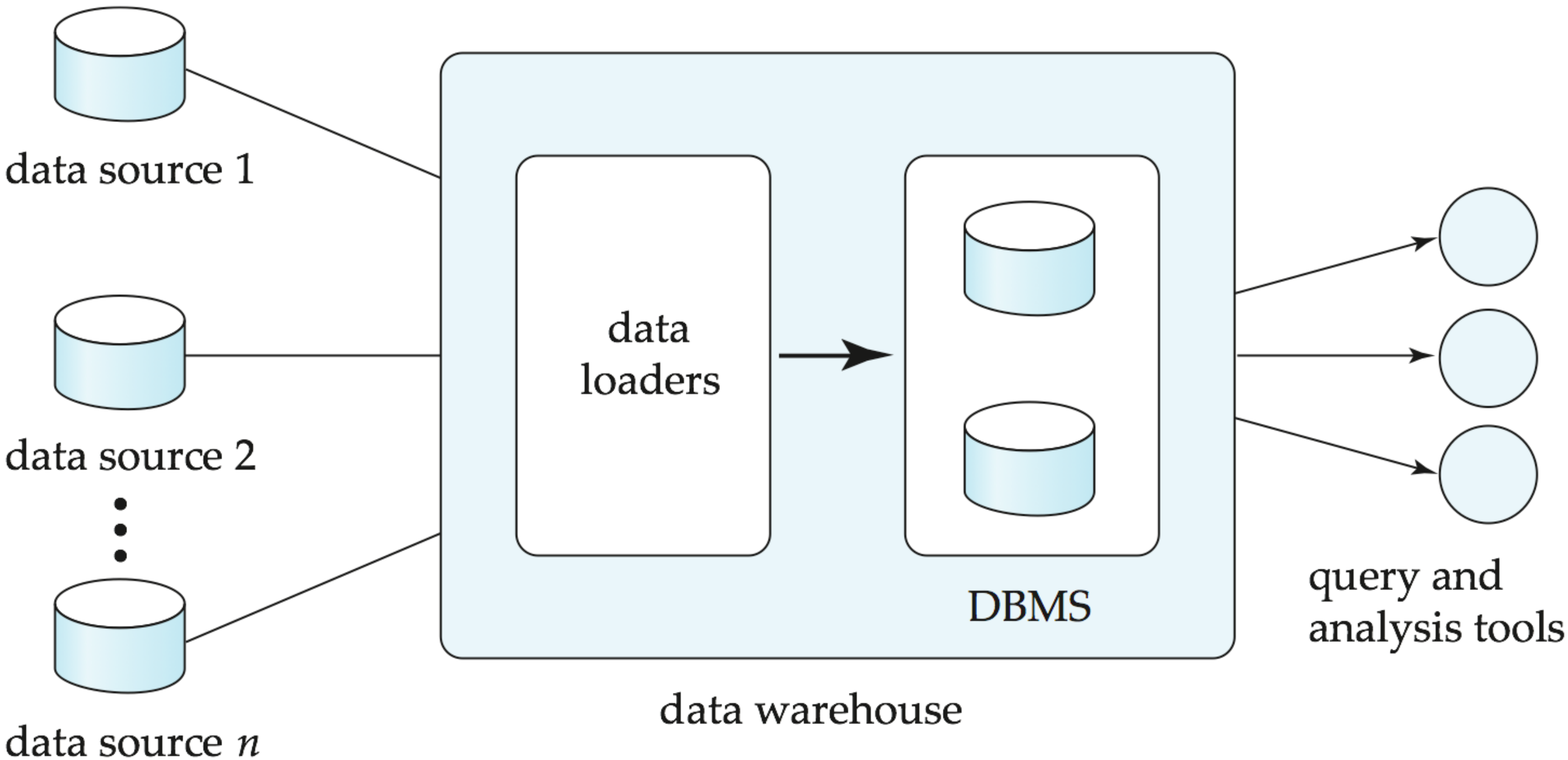
Data Warehousing

- A **data warehouse** is a repository (archive) of information gathered from multiple sources, stored under a unified schema, at a single site
  - Greatly simplifies querying, permits study of historical trends
  - Shifts decision support query load away from transaction processing systems
- Data sources often store only current data, not historical data
- Corporate decision making requires a unified view of all organizational data, including historical data

- A data warehouse is specially designed for **data analytics**, which involves reading large amounts of data to understand relationships and trends across the data. A database is used to capture and store data, such as recording details of a transaction.

Data Warehousing



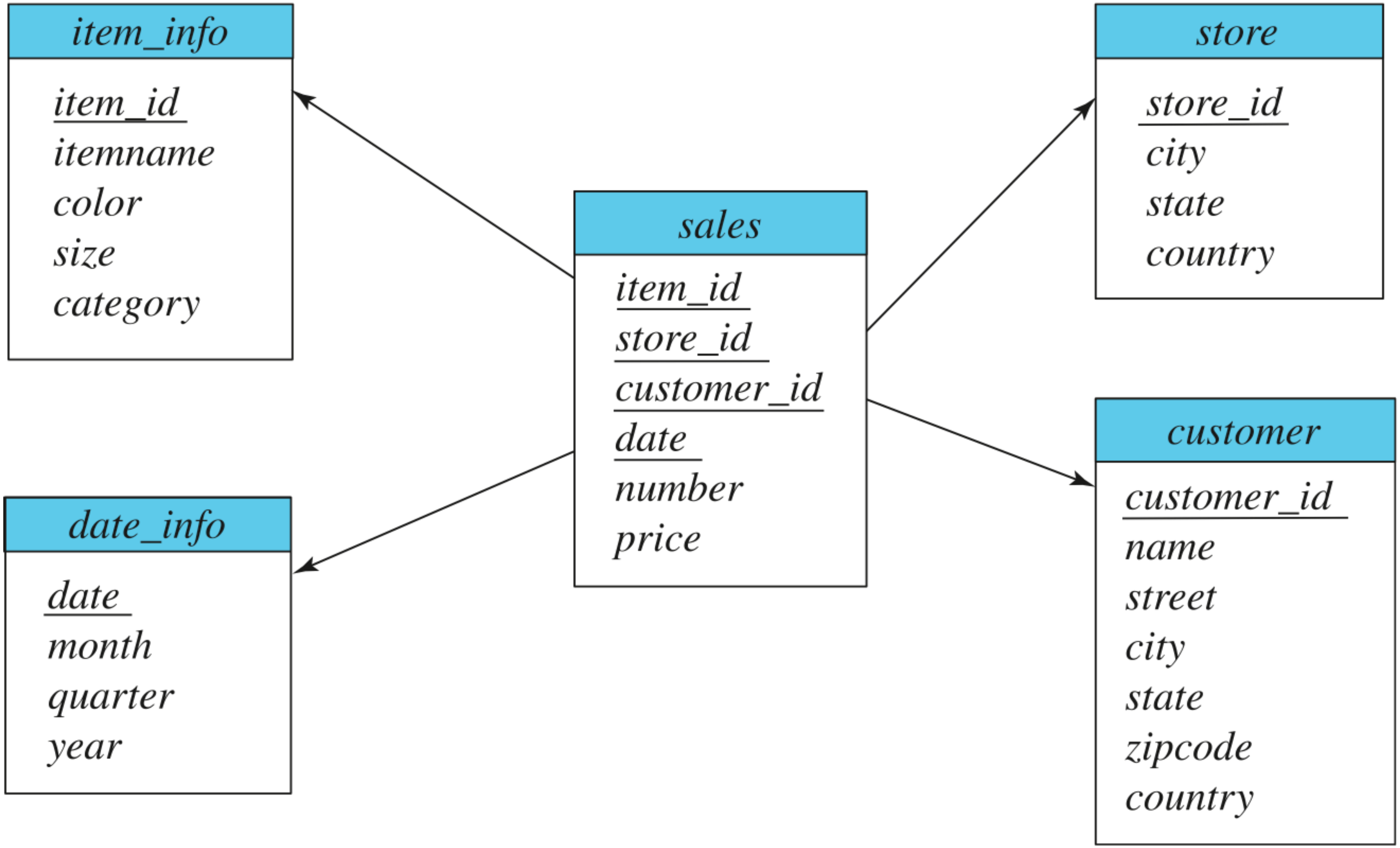
Design Issues

- *When and how to gather data*
  - **Source driven architecture**: data sources transmit new information to warehouse, either continuously or periodically (e.g., at night)
  - **Destination driven architecture**: warehouse periodically requests new information from data sources
  - Keeping warehouse exactly synchronized with data sources (e.g., using two-phase commit) is too expensive
    - Usually OK to have slightly out-of-date data at warehouse
    - Data/updates are periodically downloaded form online transaction processing (OLTP) systems.
- *What schema to use*
  - Schema integration

More Warehouse Design Issues

- *Data cleansing*
  - E.g., correct mistakes in addresses (misspellings, zip code errors)
  - **Merge** address lists from different sources and **purge** duplicates
- *How to propagate updates*
  - Warehouse schema may be a (materialized) view of schema from data sources
- *What data to summarize*
  - Raw data may be too large to store on-line
  - Aggregate values (totals/subtotals) often suffice
  - Queries on raw data can often be transformed by query optimizer to use aggregate values

Data Warehouse Schema



Data Scrubbing(data cleansing)

- Data scrubbing, also referred to as data cleansing, is the process of amending or removing data in a database that is incorrect, incomplete, improperly formatted or duplicated. Organizations in data-intensive industries like banking, insurance, retail, telecommunications and transportation use data scrubbing tools to systematically examine data for flows.
- What are common data errors that data scrubbing fixes?
- In addition to removing inaccurate or corrupt data, data scrubbing also fixes the following:
- Duplicate data. Data scrubbing is an automated way to identify duplicate records and remove them from the data set. If an organization needs to merge data from two systems, data scrubbing can be used to identify the duplicates between the two systems. The duplicate records can then be reconciled and merged to create one record representing the true data.

Data Scrubbing(Cont.)

- **Inconsistent data.** A data scrubber is a tool that analyzes data and cleans it up so that it is consistent with the rules set for that data. For example, a data scrubbing tool can help ensure that all data in a system follows a certain required format.
- **Redundant data.** Data scrubbing can help remove redundant data from your data stores, and as a result, it can reduce the amount of disk space required to store all that data.
- **General errors and typos in data.** Data scrubbing can help fix general errors such as typos and missing information. For example, while data scrubbing may not be able to fix errors in the file format, scrubbing can correct some of the errors that occur from manual entry.

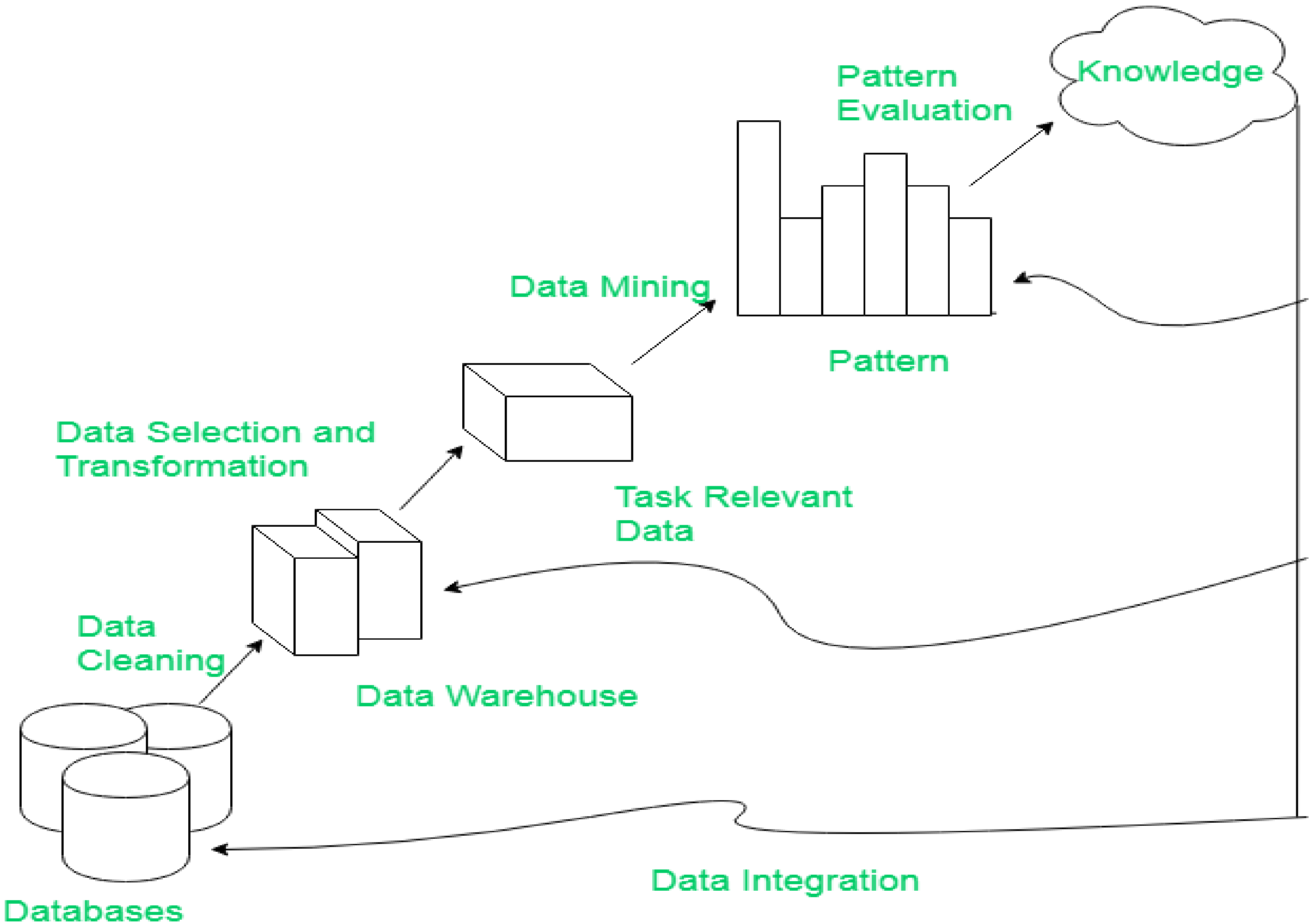
The benefits of data scrubbing

- **Improves decision-making.** Data scrubbing is an important part of the data quality process, as it identifies and removes errors, biases and inconsistencies in data. This makes the data more accurate, so it has a larger scope and can produce better results..
- **Boosts efficiency.** Data scrubbing performs data deduplication to find and remove duplicates from a database. This process matches the values of each data row in a database to that of another row and marks them as duplicate or unique.
- **Reduces inconsistencies.** Data scrubbing detects and corrects inconsistencies in data. Data inconsistencies, often called data integrity issues, can cause problems in many business processes. Poor or missing data can cost money and time and result in erroneous conclusions..

Data Mining

- Data mining is the process of semi-automatically analyzing large databases to find useful patterns
- **Prediction** based on past history
  - Predict if a credit card applicant poses a good credit risk, based on some attributes (income, job type, age, ..) and past history
  - Predict if a pattern of phone calling card usage is likely to be fraudulent
- Some examples of prediction mechanisms:
  - **Classification**
    - Given a new item whose class is unknown, predict to which class it belongs
  - **Regression** formulae
    - Given a set of mappings for an unknown function, predict the function result for a new parameter value

Knowledge Discovery in in Databases



Classification Rules

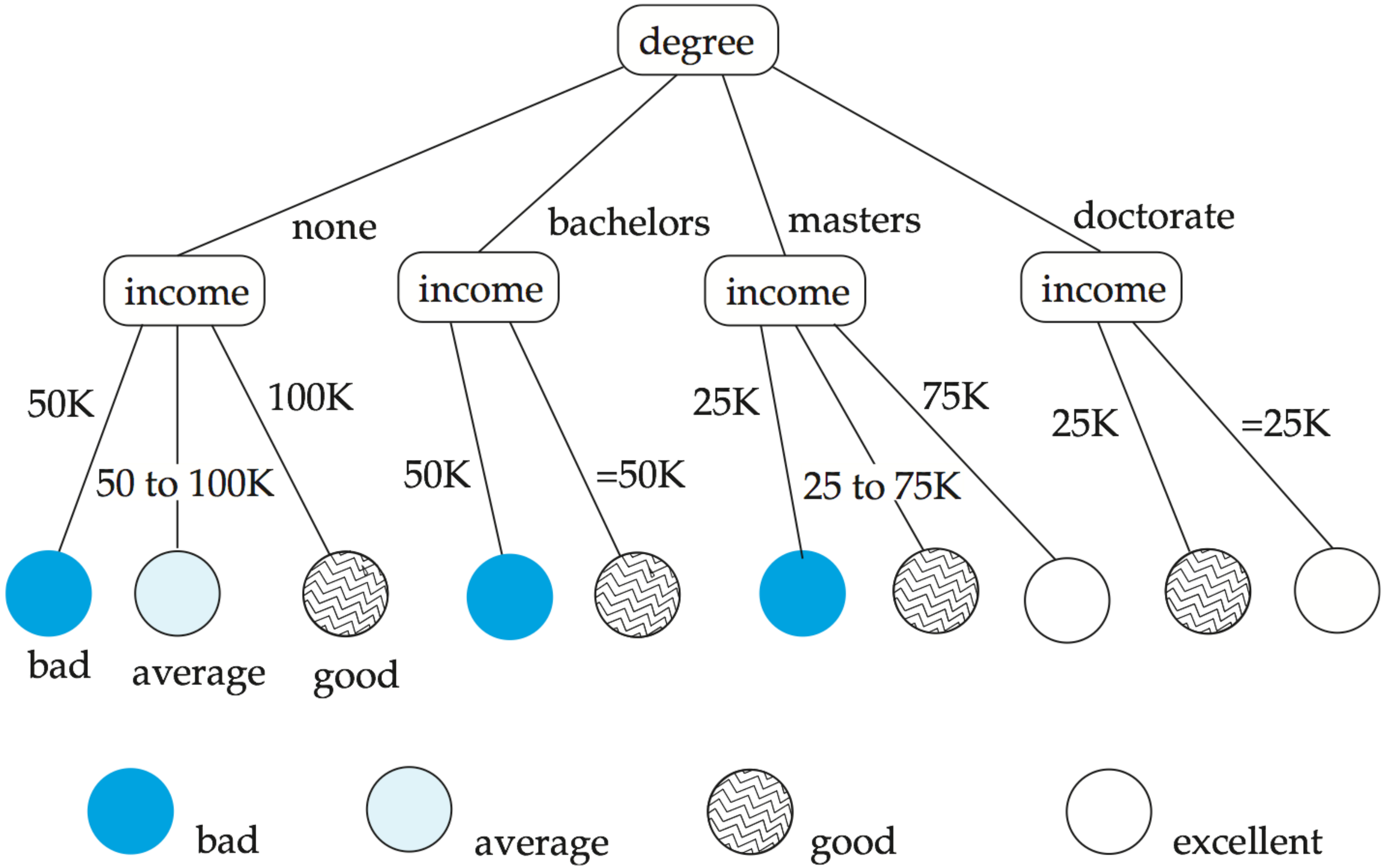
- Classification rules help assign new objects to classes.
  - E.g., given a new automobile insurance applicant, should he or she be classified as low risk, medium risk or high risk?
- Classification rules for above example could use a variety of data, such as educational level, salary, age, etc.
  - $\forall$ person P, P.degree = masters **and** P.income > 75,000

    $\Rightarrow$ P.credit = excellent
  - $\forall$ person P, P.degree = bachelors **and**

    (P.income $\geq$ 25,000 and P.income $\leq$ 75,000)

    $\Rightarrow$ P.credit = good
- Rules are not necessarily exact: there may be some misclassifications
- Classification rules can be shown compactly as a decision tree.

Decision Tree



Construction of Decision Trees

- **Training set**: a data sample in which the classification is already known.
- **Greedy** top down generation of decision trees.
  - Each internal node of the tree partitions the data into groups based on a **partitioning attribute**, and a **partitioning condition** for the node
  - **Leaf** node:
    - all (or most) of the items at the node belong to the same class, or
    - all attributes have been considered, and no further partitioning is possible.

Best Splits

- Pick best attributes and conditions on which to partition
- The purity of a set S of training instances can be measured quantitatively in several ways.
  - Notation: number of classes = *k*, number of instances = |S|, fraction of instances in class *i* = p_i.
- The **Gini** measure of purity is defined as

[

$$\text{Gini (S)} = 1 - \sum_{i- 1}^{k} p^2{}_i$$

  - When all instances are in a single class, the Gini value is 0
  - It reaches its maximum (of 1 –1 /*k*) if each class the same number of instances.

Best Splits (Cont.)

- Another measure of purity is the **entropy** measure, which is defined as

$$\text{entropy }(S) = -\sum_{i-1}^{k} p_i \log_2 p_i$$

- When a set S is split into multiple sets Si, I=1, 2, …, r, we can measure the purity of the resultant set of sets as:

$$\text{purity}(S_1, S_2, \ldots.., S_r) = \sum_{i=1}^{r} \frac{|S_i|}{|S|} \text{purity }(S_i)$$

- The information gain due to particular split of S into S_i, i = 1, 2, …., r

  Information-gain $(S, \{S_1, S_2, \ldots., S_r) = \text{purity}(S\ ) - \text{purity }(S_1, S_2, \ldots\ S_r)$

Best Splits (Cont.)

- Measure of "cost" of a split:

  Information-content $(S, \{S_1, S_2, \ldots\ldots, S_r\})) = -\sum_{i=1}^{r} \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$

- **Information-gain ratio** = $\dfrac{\text{Information-gain } (S, \{S_1, S_2, \ldots\ldots, S_r\})}{\text{Information-content } (S, \{S_1, S_2, \ldots\ldots, S_r\})}$
- The best split is the one that gives the maximum information gain ratio

Finding Best Splits

- Categorical attributes (with no meaningful order):
  - Multi-way split, one child for each value
  - Binary split: try all possible breakup of values into two sets, and pick the best
- Continuous-valued attributes (can be sorted in a meaningful order)
  - Binary split:
    - Sort values, try each as a split point
      - E.g., if values are 1, 10, 15, 25, split at ≤ 1, ≤ 10, ≤ 15
    - Pick the value that gives best split
  - Multi-way split:
    - A series of binary splits on the same attribute has roughly equivalent effect

Decision-Tree Construction Algorithm

Procedure *GrowTree* (*S*)
Partition (*S*);

Procedure Partition (*S*)
if (*purity* (*S*) > δ_p or |*S*| < δ_s) **then**
return;
for each attribute *A*
evaluate splits on attribute *A*;
Use best split found (across all attributes) to partition
S into $S_1, S_2, \ldots, S_r$,
for *i* = 1, 2, ….., *r*
Partition (S_i);

Regression

- Regression deals with the prediction of a value, rather than a class.
  - Given values for a set of variables, X_1, X_2, …, X_n, we wish to predict the value of a variable Y.
- One way is to infer coefficients a_0, a_1, a_1, …, a_n such that

  $$Y = a_0 + a_1 * X_1 + a_2 * X_2 + \ldots + a_n * X_n$$

- Finding such a linear polynomial is called **linear regression**.
  - In general, the process of finding a curve that fits the data is also called **curve fitting**.
- The fit may only be approximate
  - because of noise in the data, or
  - because the relationship is not exactly a polynomial
- Regression aims to find coefficients that give the best possible fit.

Association Rules

- Retail shops are often interested in associations between different items that people buy.
  - Someone who buys bread is quite likely also to buy milk
  - A person who bought the book *Database System Concepts* is quite likely also to buy the book *Operating System Concepts*.
- Associations information can be used in several ways.
  - E.g., when a customer buys a particular book, an online shop may suggest associated books.
- **Association rules**:

  bread $\Rightarrow$ *milk*   *DB-Concepts, OS-Concepts* $\Rightarrow$ Networks

  - Left hand side: **antecedent**,   right hand side: **consequent**
  - An association rule must have an associated **population**; the population consists of a set of **instances**
    - E.g., each transaction (sale) at a shop is an instance, and the set of all transactions is the population

Association Rules (Cont.)

- Rules have an associated support, as well as an associated confidence.
- **Support** is a measure of what fraction of the population satisfies both the antecedent and the consequent of the rule.
  - E.g., suppose only 0.001 percent of all purchases include milk and screwdrivers. The support for the rule is *milk* ⇒ *screwdrivers* is low.
- **Confidence** is a measure of how often the consequent is true when the antecedent is true.
  - E.g., the rule *bread* ⇒ *milk* has a confidence of 80 percent if 80 percent of the purchases that include bread also include milk.

Finding Association Rules

- We are generally only interested in association rules with reasonably high support (e.g., support of 2% or greater)
- Naïve algorithm
  1. Consider all possible sets of relevant items.
  2. For each set find its support (i.e., count how many transactions purchase all items in the set).
     - **Large itemsets**: sets with sufficiently high support
  3. Use large itemsets to generate association rules.
     1. From itemset A generate the rule $A - \{b\} \Rightarrow b$ for each $b \in A$.
        - Support of rule = support (A).
        - Confidence of rule = support (A) / support ($A - \{b\}$)

Finding Support

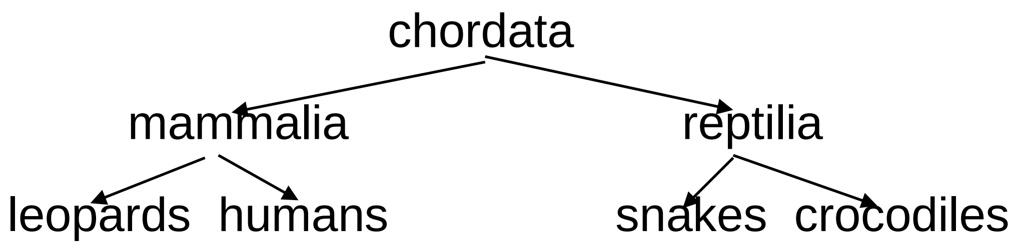
- Determine support of itemsets via a single pass on set of transactions
  - Large itemsets: sets with a high count at the end of the pass
- If memory not enough to hold all counts for all itemsets use multiple passes, considering only some itemsets in each pass.
- Optimization: Once an itemset is eliminated because its count (support) is too small none of its supersets needs to be considered.
- The **a priori** technique to find large itemsets:
  - Pass 1: count support of all sets with just 1 item. Eliminate those items with low support
  - Pass *i*: **candidates**: every set of *i* items such that all its *i-1* item subsets are large
    - Count support of all candidates
    - Stop if there are no candidates

Clustering

- Clustering: Intuitively, finding clusters of points in the given data such that similar points lie in the same cluster
- Can be formalized using distance metrics in several ways
  - Group points into *k* sets (for a given *k*) such that the average distance of points from the centroid of their assigned group is minimized
    - Centroid: point defined by taking average of coordinates in each dimension.
  - Another metric: minimize average distance between every pair of points in a cluster
- Has been studied extensively in statistics, but on small data sets
  - Data mining systems aim at clustering techniques that can handle very large data sets
  - E.g., the Birch clustering algorithm (more shortly)

Hierarchical Clustering

- Example from biological classification
  - (the word classification here does not mean a prediction mechanism)



- Other examples: Internet directory systems (e.g., Yahoo, more on this later)
- **Agglomerative clustering algorithms**
  - Build small clusters, then cluster small clusters into bigger clusters, and so on
- **Divisive clustering algorithms**
  - Start with all items in a single cluster, repeatedly refine (break) clusters into smaller ones

Clustering Algorithms

- Clustering algorithms have been designed to handle very large datasets
- E.g., the **Birch algorithm**
  - Main idea: use an in-memory R-tree to store points that are being clustered
  - Insert points one at a time into the R-tree, merging a new point with an existing cluster if is less than some δ distance away
  - If there are more leaf nodes than fit in memory, merge existing clusters that are close to each other
  - At the end of first pass we get a large number of clusters at the leaves of the R-tree
    - Merge clusters to reduce the number of clusters

Collaborative Filtering

- Goal: predict what movies/books/… a person may be interested in, on the basis of
  - Past preferences of the person
  - Other people with similar past preferences
  - The preferences of such people for a new movie/book/…
- One approach based on repeated clustering
  - Cluster people on the basis of preferences for movies
  - Then cluster movies on the basis of being liked by the same clusters of people
  - Again cluster people based on their preferences for (the newly created clusters of) movies
  - Repeat above till equilibrium
- Above problem is an instance of **collaborative filtering**, where users collaborate in the task of filtering information to find information of interest

Information Retrieval Systems

- **Information retrieval (IR)** systems use a simpler data model than database systems
  - Information organized as a collection of documents
  - Documents are unstructured, no schema
- Information retrieval locates relevant documents, on the basis of user input such as keywords or example documents
  - e.g., find documents containing the words "database systems"
- Can be used even on textual descriptions provided with non-textual data such as images
- Web search engines are the most familiar example of IR systems

Information Retrieval Systems (Cont.)

- Differences from database systems
  - IR systems don't deal with transactional updates (including concurrency control and recovery)
  - Database systems deal with structured data, with schemas that define the data organization
  - IR systems deal with some querying issues not generally addressed by database systems
    - Approximate searching by keywords
    - Ranking of retrieved answers by estimated degree of relevance

Keyword Search

- In **full text** retrieval, all the words in each document are considered to be keywords.
  - We use the word **term** to refer to the words in a document
- Information-retrieval systems typically allow query expressions formed using keywords and the logical connectives *and, or,* and *not*
  - *And*s are implicit, even if not explicitly specified
- Ranking of documents on the basis of estimated relevance to a query is critical
  - Relevance ranking is based on factors such as
    - Term frequency
      - Frequency of occurrence of query keyword in document
    - Inverse document frequency
      - How many documents the query keyword occurs in
        - Fewer ➔ give more importance to keyword
    - Hyperlinks to documents
      - More links to a document ➔ document is more important

Application of Information Retrieval

- Digital library :

  A digital library is a library in which collections are stored in digital formats and accessible by computers. The digital content may be stored locally, or accessed remotely via computer networks. A digital library is a type of information retrieval system.

- Search Engines:

  A search engine is one of the most the practical applications of information retrieval techniques to large scale text collections. Web search engines are best- known examples, but many others searches exist, like: Desktop search, Enterprise search, Federated search, Mobile search, and Social search.

- Media Search:

  An image retrieval system is a computer system for browsing, searching and retrieving image from a large database of digital images